



Programming

Advanced Python Fun: Part I

Vassilis Markos, Mediterranean College

Week 10

Contents



1 args And kwargs

2 Fun Time!

args **And** kwargs

Challenge

Define a function `foo()` that behaves as shown below (DO NOT inspect the contents of `foo.py`).

```
1 # source/boom_001.py
2
3 from foo import foo
4
5 if __name__ == "__main__":
6     foo()
7     foo(7)
8     foo(5, 9)
9     foo(4, 3, 1)
10    foo(5, 2.02, "haha", "hohoho!")
```

foo



Let us have a look at `foo()`:

```
1 # source/foo.py
2
3 def foo(*args):
4     if not args:
5         print("Oops! No arguments!")
6         return
7     for i, arg in enumerate(args):
8         print(f"{i}: {arg}", end=" ")
9     print()
```

`*args`



- `*args` is a nice and compact way python allows us to pass positional arguments to a function.
- Roughly, you can think of `*args` arguments as a tuple (immutable list) of values that is passed as a single argument into a list.
- Essentially, this means that we can provide a function with an arbitrary number of arguments, without having to define multiple versions of it!
- Order matters, in the sense that the contents of `*args` are spread in the same order they are provided to the function!

What Will This Print?

```
1 # source/boom_002.py
2
3 def bar(a, b, *args):
4     if not args:
5         return a + b
6     return sum(args)
7
8 if __name__ == "__main__":
9     print(bar(1, 2))
10    print(bar(1, 3, 2))
11    print(bar(1))
12    print("What?")
```

Be Careful!

The above should print something like that:

```
1 3
2 2
3 Traceback (most recent call last):
4   File "boom_002.py", line 11, in <module>
5     print(bar(1))
6 TypeError: bar() missing 1 required positional argument: 'b'
```

Be Careful!

The above should print something like that:

```
1 3
2 2
3 Traceback (most recent call last):
4   File "boom_002.py", line 11, in <module>
5     print(bar(1))
6 TypeError: bar() missing 1 required positional argument: 'b'
```

- While `*args` might be an empty tuple, if we also define other positional arguments, we have to pass them no matter what!

Be Careful!

The above should print something like that:

```
1 3
2 2
3 Traceback (most recent call last):
4   File "boom_002.py", line 11, in <module>
5     print(bar(1))
6 TypeError: bar() missing 1 required positional argument: 'b'
```

- While `*args` might be an empty tuple, if we also define other positional arguments, we have to pass them no matter what!
- So, in this case, `bar()` should always be called with **at least two arguments!**

Antoher Challenge



Define a function `baz()` that behaves as shown below (DO NOT inspect the contents of `baz.py`).

```
1 # source/boom_003.py
2
3 from baz import baz
4
5 if __name__ == "__main__":
6     baz(1)
7     baz(4, msg="Look!")
8     baz(3, message="Look!")
9     baz(5, name="Alice", msg="Well...")
```

baz()

```
1 # source/baz.py
2
3 def baz(num, **kwargs):
4     name = "John Doe"
5     msg = "My Number is: "
6     if "name" in kwargs:
7         name = kwargs["name"]
8     if "msg" in kwargs:
9         msg = kwargs["msg"]
10    known_kwargs = {"name", "msg"}
11    for kwarg in set(kwargs).difference(known_kwargs):
12        print(f"Unknown argument: {kwarg}.")
13    print(f"{name} says: '{msg} {num}'.")
```

Keyword Arguments



- Much like list arguments, keyword arguments are arguments provided using keywords, in the same form as a Python dictionary.
- The same restrictions to spread positional arguments apply, i.e.:
 - We can provide an arbitrary number of keyword arguments.
 - We can iterate them in the same way we do with dictionaries.
 - Order does not matter, since we are passing arguments by keyword.
- That way, one can pass named arguments to a function, using the same names as in the function's definition.

*args: The Other Way Around

We can also use *args to spread list contents as arguments into a list, by using the list spread operator, *.

```
1 # source/boom_004.py
2
3 def add(a, b, c):
4     return a + b + c
5
6 if __name__ == "__main__":
7     xs = [3, 1, 2]
8     print(add(xs[0], xs[1], xs[2])) # Old-school
9     print(add(*xs)) # Pythonic
```

****kwargs: The Other Way Around**



We can also use `**kwargs` to spread dict contents as arguments, by using the dictionary spread operator, `**`.

```
1 # source/boom_005.py
2
3 def add(a, b, c):
4     return a + b + c
5
6 if __name__ == "__main__":
7     xs = {'a': 4, 'b': 6, 'c': 2}
8     print(add(a=xs['a'], b=xs['b'], c=xs['c'])) # Old-school
9     print(add(**xs)) # Pythonic
```

Some Remarks



- (Reminder) You cannot use a positional argument after a keyword one!
- So, you have to collect all postional arguments at the beginning and all keyword arguments at the end of the function parameters list.
- This also extends to `*args` and `**kwargs`, which means that we cannot expect for positional arguments after using `**kwargs` — in the same spirit as above.
- One common use of `**kwargs` is to provide default values to keyword arguments and then allow users to provide custom values whenever they want to.



Fun Time!

In-class Exercise #001: Revision (Arrays)



Solve all exercises found in:

`labs/arrays_lab/arrays_01.pdf`

Pay attention to the subtle points of each exercise and what you should compute in each time (especially in the two last ones).

In-class Exercise #002: Revision (Files)



Solve all exercises found in:

`labs/files_lab/files_01.pdf`

In all cases, do not hardcode any paths and use appropriate Python libraries to handle proper path finding.

In-class Exercise #003: Revision (Classes)



Solve all exercises found in:

`labs/classes_lab/objects_01.pdf`

As with the two previous ones, stay calm and pay attention to details.
Also, make your designs as succinct as possible!

Any Questions?

Do not forget to fill in
the questionnaire shown
right!

Office Hours

Glyfada: Tuesday, 09:00–10:00, PC Lab
Pellinis: Wednesday, 15:00–17:00, 203



<https://forms.gle/dKSrmE1VRVWqxBGZA>